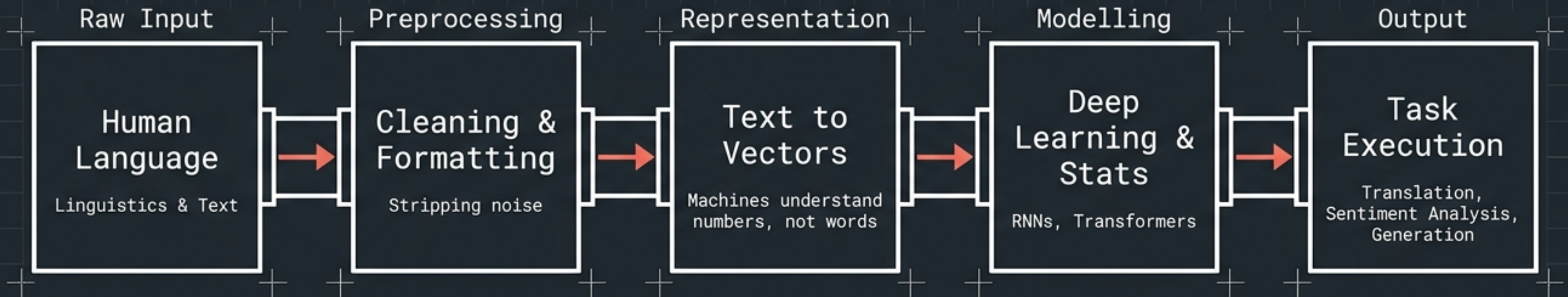


From Word Counting to Generative AI: The Technical Evolution of Natural Language Processing



Structuring the Unstructured: The NLP Pipeline



NLP combines Linguistics, Machine Learning, Deep Learning, and Statistics to enable computers to process and generate human language.

Phase 1: Preparing the Raw Material

Bite-Sized Modular Cards

Lowercasing

Reduces vocabulary size.

'Apple' → 'apple'

Tokenisation

Splitting text into units
(Words, Characters, or Subwords
like BPE/SentencePiece).

Stopword Removal

Deleting low-value structural words.

'is', 'the', 'a'

The Roots Matrix

Stemming

- Crude root reduction. Cuts off suffixes.
- Fast but creates invalid words.
- Algorithms: Porter, Snowball

'Caring' → 'Car'

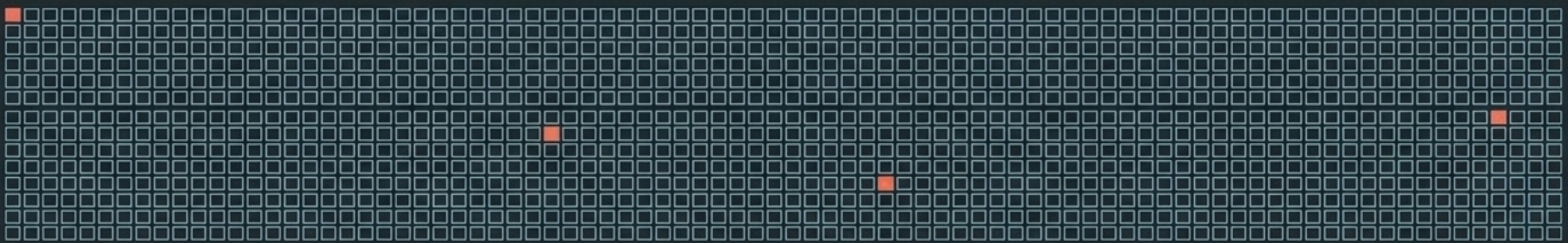
Lemmatisation

- Converts to meaningful dictionary root.
- Slower but highly accurate.
- Used for vocabulary normalisation.

'Caring' → 'Care'

Phase 2: The Transition to Mathematics

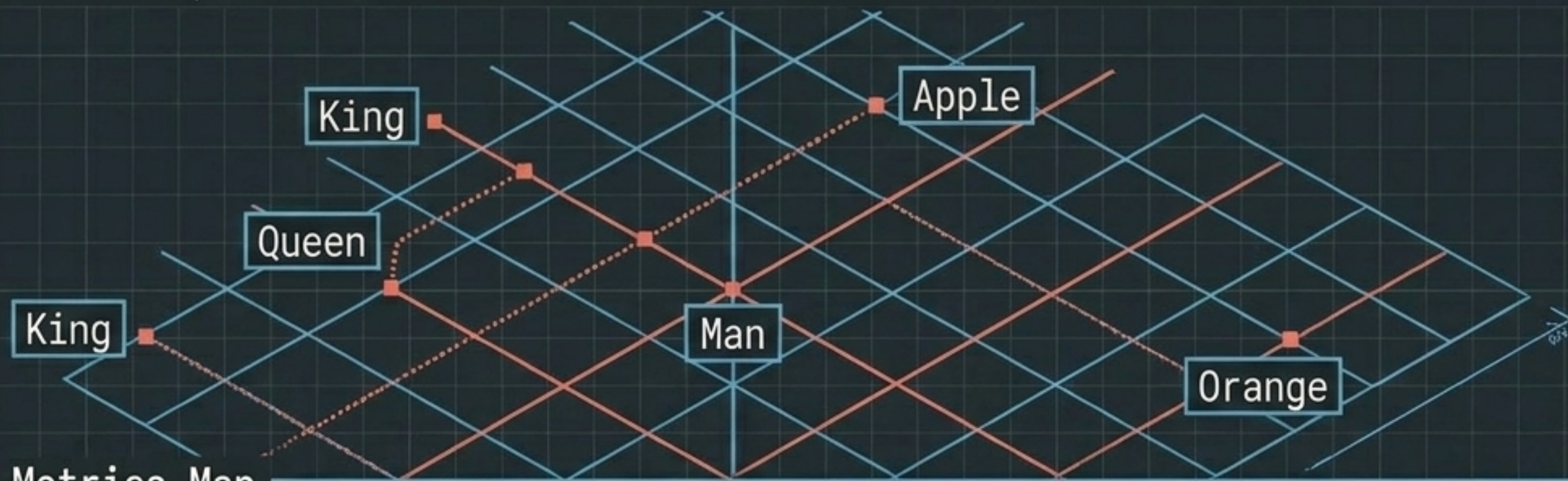
The Sparsity Problem



Bag of Words (BoW)	N-Grams (Unigram, Bigram, Trigram)	TF-IDF (Term Frequency - Inverse Document Frequency)
Mechanism: Word frequency count. Flaw: Ignores meaning and word order. Creates massive, sparse vectors.	Mechanism: Captures word sequences (e.g., pairs or triplets). Advantage: Captures localised context. Flaw: Vocabulary explosion (dimensions grow exponentially).	Mechanism: Formula weighing how often a word appears vs. how rare it is globally. Advantage: Highlights truly important words, penalising common filler.

The Vector Space Breakthrough: Word Embeddings

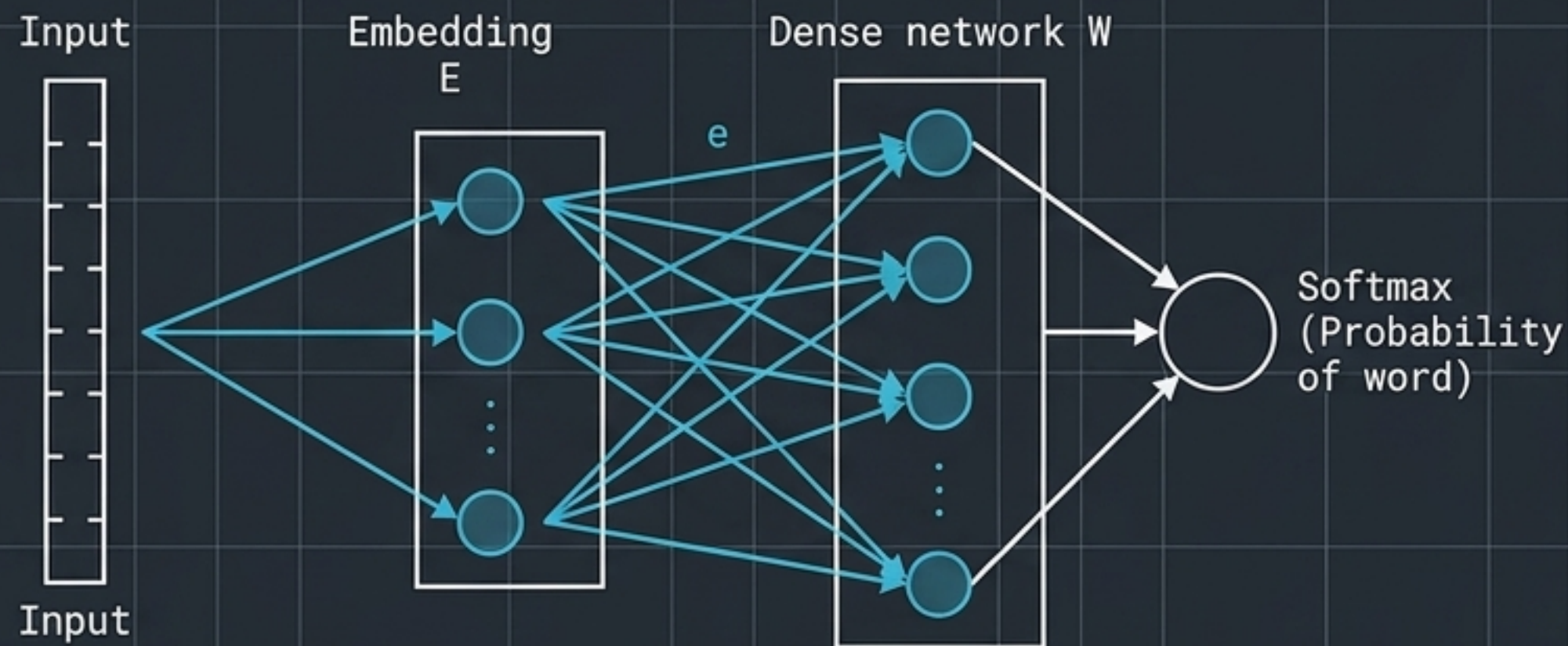
Dense vector representations mapping words into a continuous, lower-dimensional space (e.g., 300 dimensions) where similar words have similar numerical values.



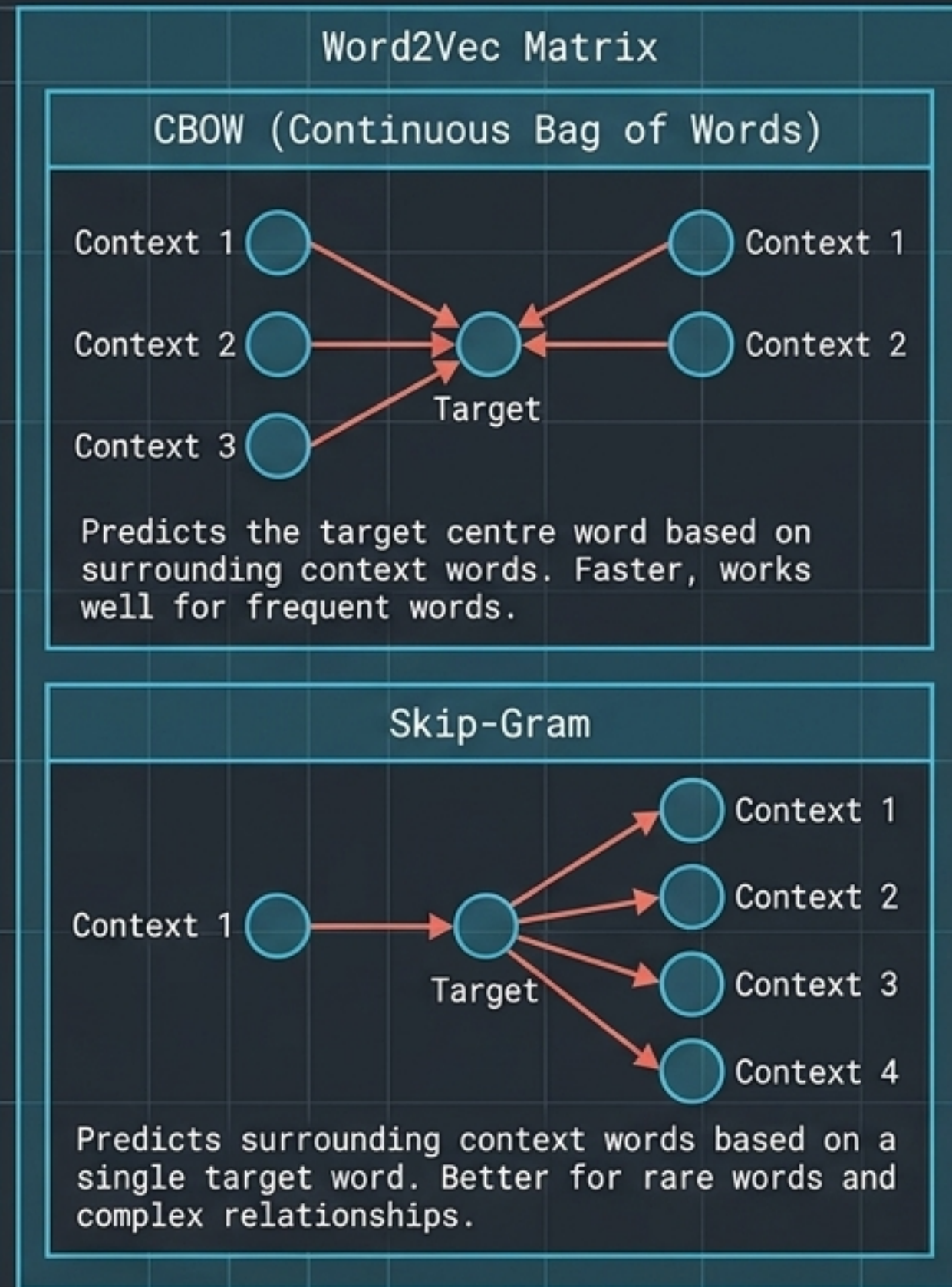
Distance Metrics Map

Cosine Similarity	Euclidean	Manhattan	Jaccard
$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\ \mathbf{A}\ \ \mathbf{B}\ }$ Measures angle; 1 = identical, -1 = opposite	Straight line distance.	Grid path distance.	Intersection over union.

Featurized Representation: Generating Embeddings



The network learns a weight matrix E (e.g., 300 x 10,000 dimensions). The hidden layer weights learned during the task of predicting the next word become the permanent embedding vector for that word.



The Sequence Bottleneck: Memory and Time

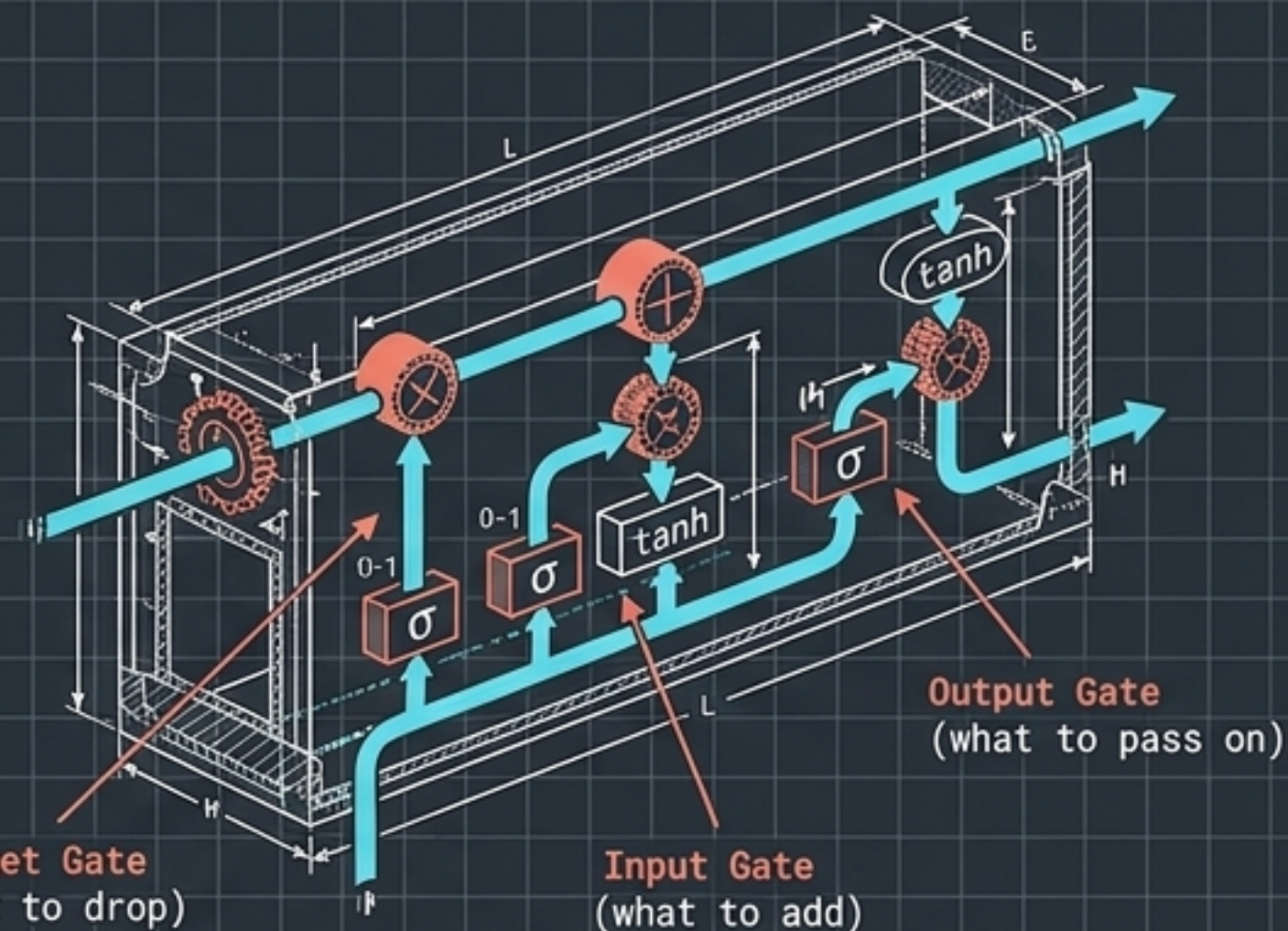
RNN Block



Mechanism: Processes tokens sequentially, maintaining a hidden state.

The Fatal Flaw: The Vanishing Gradient. Gradients become microscopically small during backpropagation, causing the model to forget long-term context.

LSTM (Long Short-Term Memory) Block



Mechanism: Uses an internal memory highway with regulatory gates.

Anatomy: Forget Gate (what to drop) | Input Gate (what to add) | Output Gate (what to pass on).

Result: Solves vanishing gradients, enabling translation and speech recognition. (Note: GRU is a simpler, faster variant with only Update/Reset gates).

The Paradigm Shift: The Attention Mechanism

Concept: Instead of processing words in strict order, the model learns to selectively focus on the most relevant surrounding tokens, regardless of distance.



1. Query (Q)

What the current token is looking for.

2. Key (K)

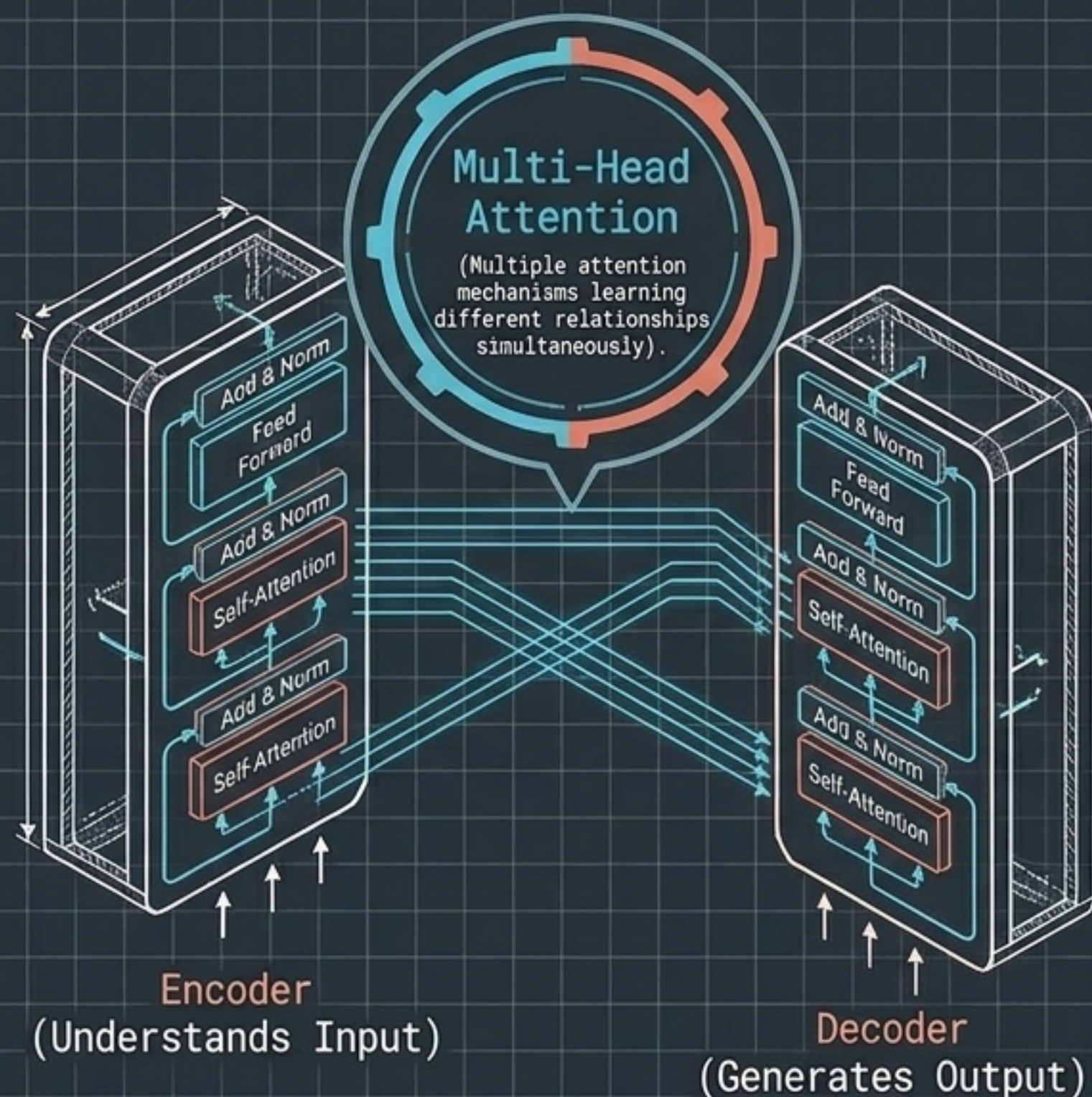
What other tokens have to offer.

3. Value (V)

The actual content of the token.

Process: Calculates similarity scores between Q and K → applies Softmax → computes a weighted sum of V.

The Architecture of Modern AI: Transformers



The RNN vs. Transformer Breakdown

RNNs:

Sequential processing
(Token 1 → Token 2 → Token 3).
Slow to train, struggles with extreme long-range dependencies.

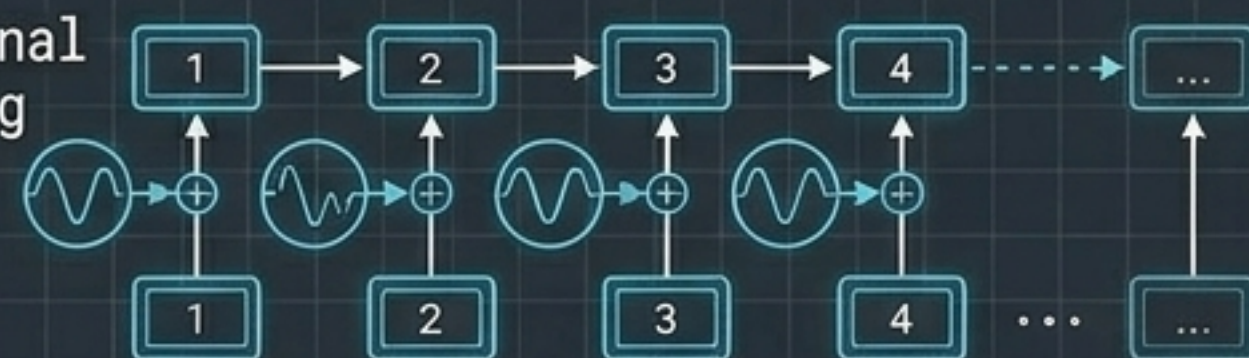
Transformers:

Parallel processing.
Computes relationships between every pair of tokens simultaneously.
Scales quadratically.

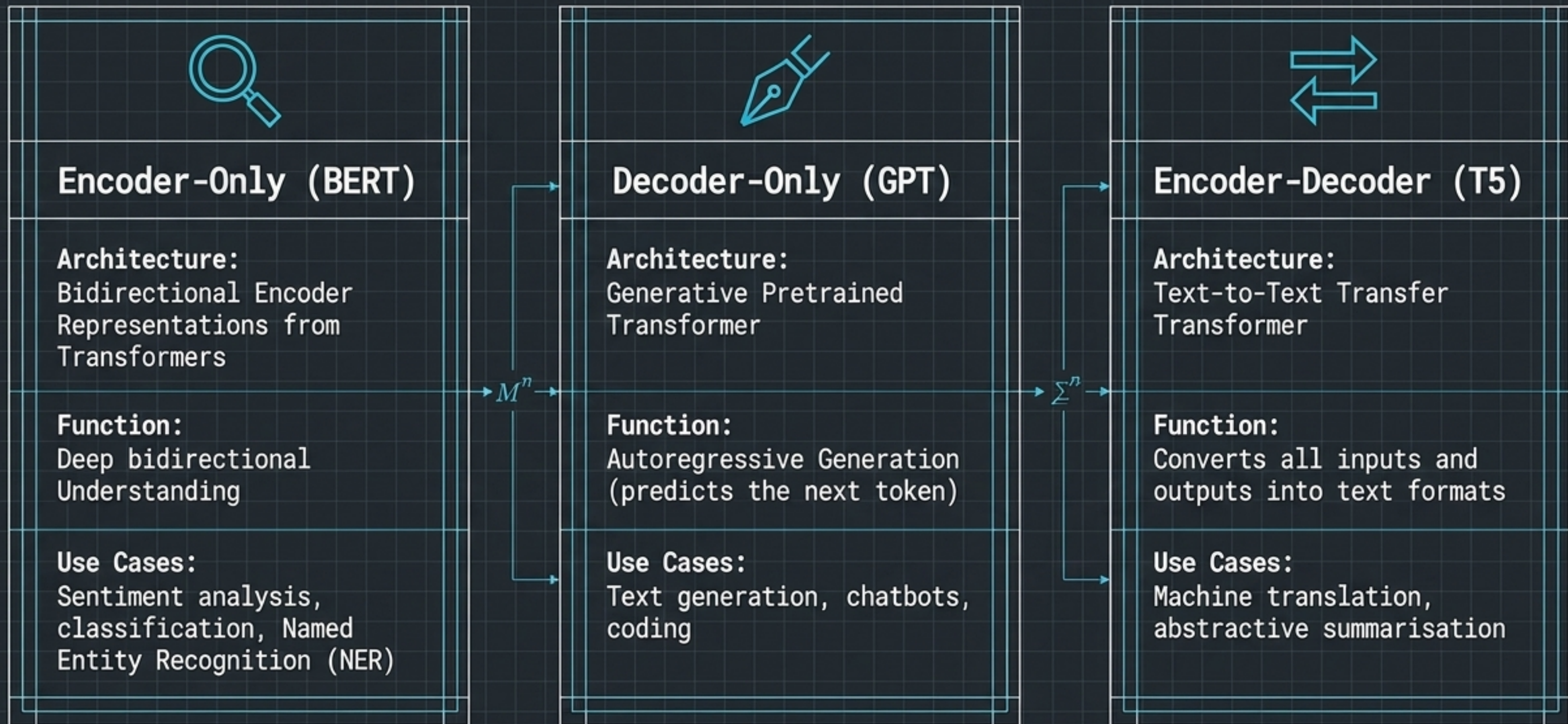
Key Innovation: Positional Encoding

Because Transformers process everything in parallel, they lack native sequence awareness. Positional encoding mathematically injects sequence order information back into the tokens.

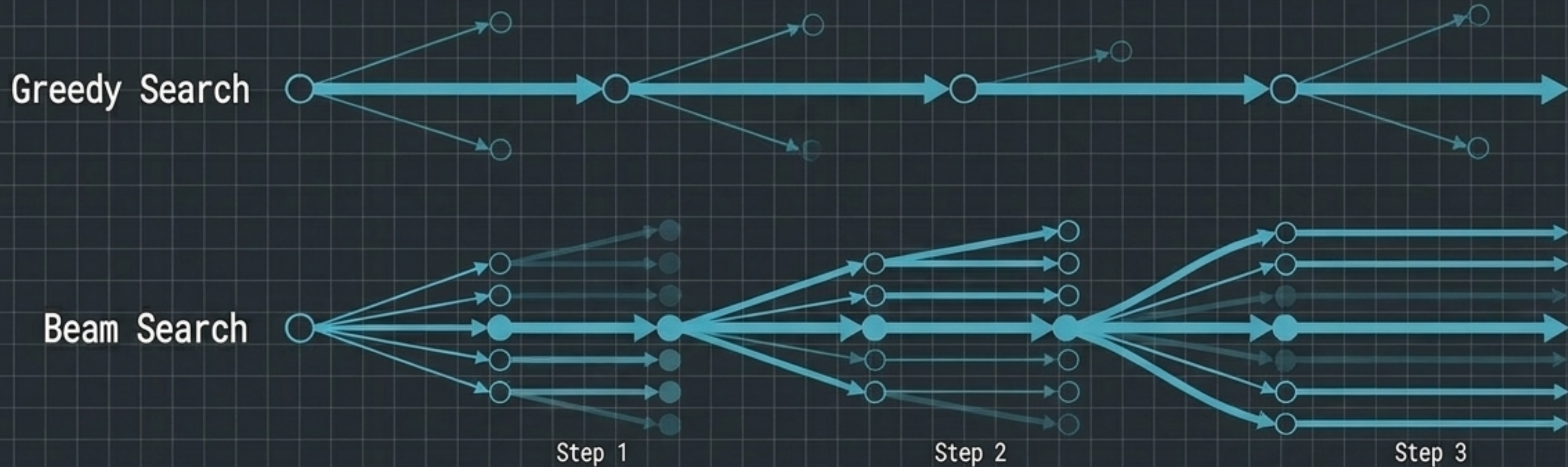
Positional Encoding



The LLM Trinity: Structuring the Giants



The Generation Engine: Search & Temperature

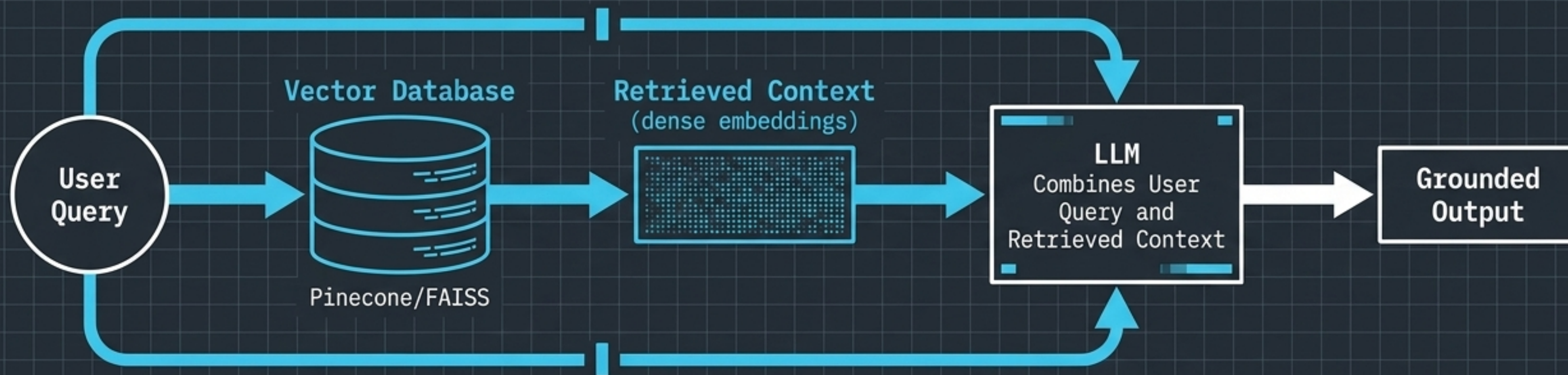


Decoding Strategies	Controlling Output (Temperature)
Greedy Search: Chooses the single best immediate word. Fast, but often produces suboptimal full sentences.	Low Temperature: Highly deterministic, focused, predictable text.
Beam Search: Explores multiple sequence possibilities (top-k) simultaneously to find the highest-quality overall output.	High Temperature: Increases randomness, producing creative or highly varied outputs. Measured by Perplexity (lower is better).

Grounding the AI: Solving Hallucination with RAG

The Problem: Hallucination – when an LLM generates false information with high statistical confidence.

The Solution: Retrieval-Augmented Generation (RAG).



1. Store:

Embed custom knowledge into a Vector Database (e.g., Pinecone, ChromaDB).

2. Retrieve:

Match the user query to relevant documents using **Cosine Similarity**.

3. Augment:

Inject the retrieved facts into the LLM's **Context Window** (the maximum tokens it can process, e.g., 128k).

4. Generate:

The LLM synthesises an answer strictly based on the provided **factual context**.

Engineering Efficiency: Compression and Adaptation



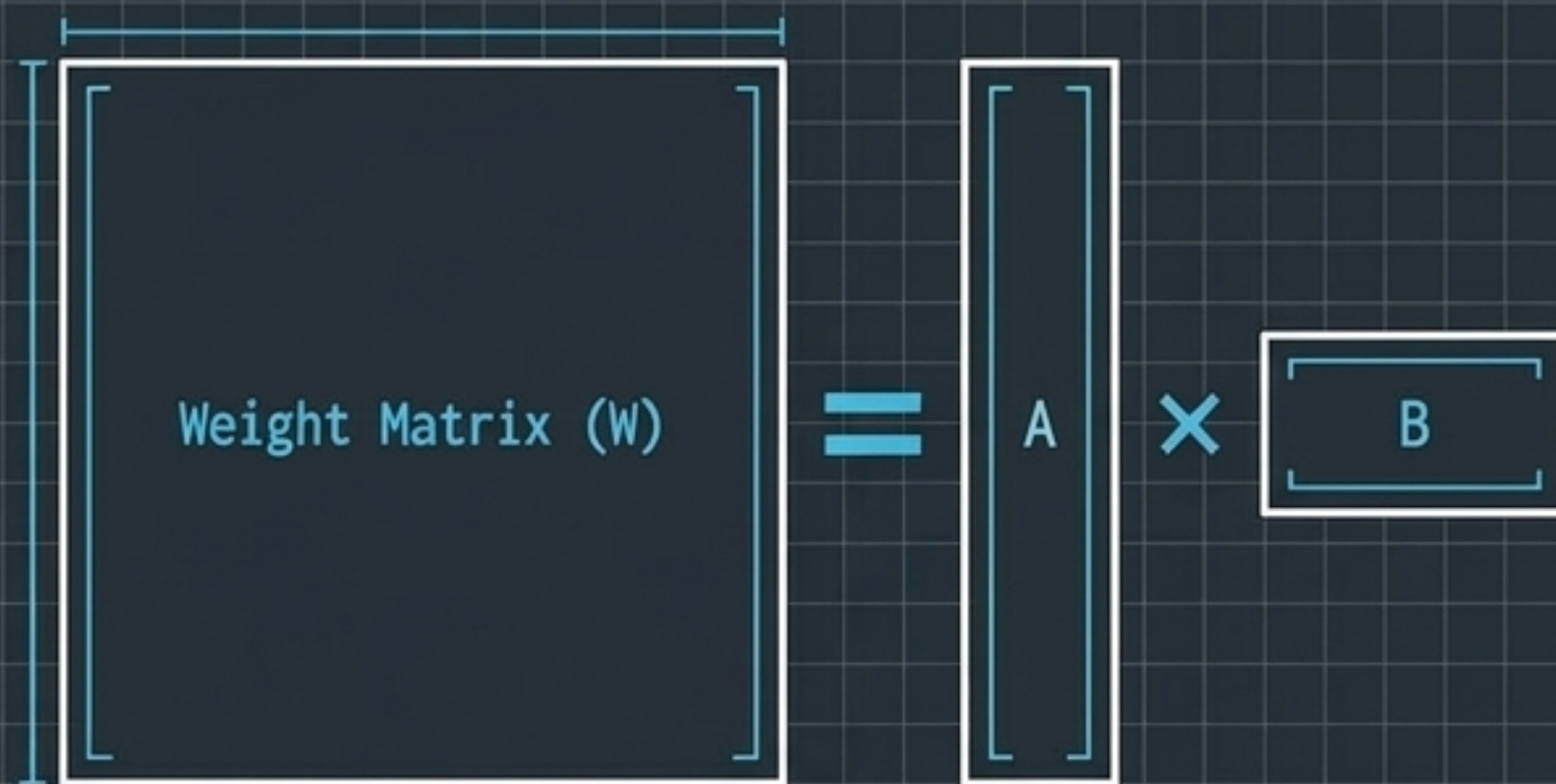
Quantisation:

Concept:

Reducing the precision of the model's numbers (e.g., 32-bit floating point $\rightarrow$ 8-bit integer).

Result:

Drastically shrinks model size and increases inference speed with minimal loss in reasoning quality.



LoRA (Low-Rank Adaptation):

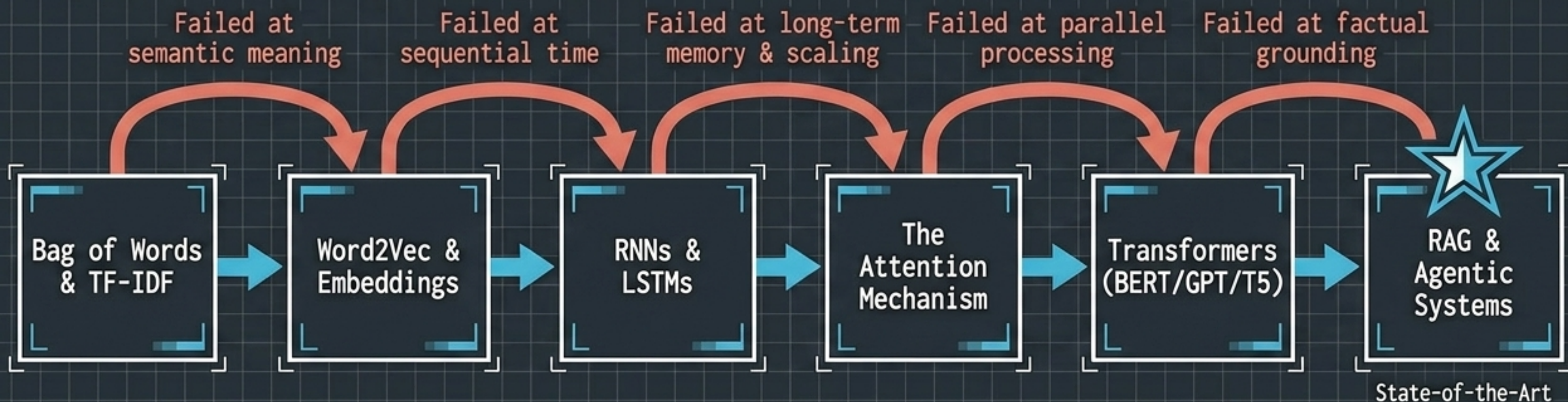
Concept:

Parameter-efficient fine-tuning. Instead of retraining the entire massive network, LoRA injects **small, low-rank matrices** into the Transformer architecture.

Result:

Slashes **GPU memory requirements** and accelerates training for domain adaptation.

Synthesis I: The Evolution of Understanding



Synthesis II: The Master NLP Diagnostic Matrix

Concept	Core Idea	Primary Advantage / Tradeoff
Tokenisation	Splitting text into subwords (BPE).	Essential first step.
Embeddings	Dense semantic vectors.	Captures meaning; allows math on language.
LSTM vs RNN	Sequence modelling with gates.	Solves vanishing gradients .
Attention	Similarity scores between tokens.	Focuses only on important words.
Transformers	Parallel, attention-based architecture.	Highly scalable ; SOTA performance.
BERT vs GPT	Understanding (Encoder) vs. Generation (Decoder).	Task-specific architectures.
RAG	Retrieval + Generation.	Eliminates hallucinations.
BLEU vs ROUGE	Translation quality vs. Summarisation quality metrics.	Standardised evaluation.
LoRA	Low-rank matrix fine-tuning.	Cheap, fast domain adaptation.